

МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ

Факультет Информационных технологий  
Кафедра Информатики и информационных технологий

направление подготовки  
09.03.02 «Информационные системы и технологии»

ПРАКТИЧЕСКАЯ РАБОТА № 14

Дисциплина: «BackEnd-разработка»

Тема: *Работа с кросс-доменными запросами веб-приложение на основе  
ASP.NET Core*

Выполнил(а): студент(ка) группы 241-333

Конев Максим Дмитриевич  
(Фамилия И.О.)

Дата, подпись \_\_\_\_\_  
(Дата) (Подпись)

Проверил: \_\_\_\_\_  
(Фамилия И.О., степень, звание) (Оценка)

Дата, подпись \_\_\_\_\_  
(Дата) (Подпись)

Москва  
2026

## **Цель работы:**

Ознакомиться с настройкой и использованием механизма CORS для обработки кросс-доменных запросов в приложениях на платформе ASP.NET Core.

## **Задание:**

1. Создать проект ASP.NET Core Web API или использовать ранее созданный проект.
2. Настроить поддержку кросс-доменных запросов в приложении с использованием CORS.
3. Реализовать одну или несколько политик CORS с различными параметрами доступа.
4. Настроить разрешения для источников, HTTP-методов и заголовков запросов.
5. Проверить работу кросс-доменных запросов при обращении к API из внешнего клиента или браузера.
6. Сравнить результаты работы приложения при включённой и отключённой настройке CORS.

## **1. Написание приложения**

1. **Использование существующего проекта.** В качестве основы для работы использован проект StoreApi из лабораторной работы №12, реализующий REST API для интернет-магазина. Проект построен на платформе **ASP.NET Core 10** с использованием базы данных **MySQL**. Схема базы данных включает пять связанных сущностей: Category, Product, Customer, Order, OrderItem.
2. **Архитектура CORS.** В проекте реализована двухуровневая система политик CORS для поддержки разработки и продакшена:
  - AllowAll (Development)
    1. AllowAnyOrigin/Method/Header
    2. Для локального тестирования
  - AllowFrontend (Production)

1. WithOrigins(...)
2. WithMethods(GET,POST,...)
3. WithHeaders(...)
4. AllowCredentials()

3. **Регистрация политик CORS.** В Program.cs добавлена регистрация двух ПОЛИТИК:

```
builder.Services.AddCors(options =>
{
    // Политика для разработки
    options.AddPolicy("AllowAll", policy =>
    {
        policy.AllowAnyOrigin()
            .AllowAnyMethod()
            .AllowAnyHeader();
    });

    // Политика для продакшена
    options.AddPolicy("AllowFrontend", policy =>
    {
        policy.WithOrigins("http://localhost:8000", "https://myshop-frontend.com")
            .WithMethods("GET", "POST", "PUT", "DELETE", "OPTIONS")
            .WithHeaders("Content-Type", "Authorization", "X-Requested-With")
            .AllowCredentials();
    });
});
```

4. **Подключение middleware CORS.** Middleware UseCors подключён в конвейер обработки запросов **строго после UseRouting и перед UseAuthorization:**

```
var app = builder.Build();

app.UseRouting();

if (app.Environment.IsDevelopment())
{
    app.UseCors("AllowAll");
    app.UseExceptionHandler("/error-development");
    app.UseSwagger();
    app.UseSwaggerUI();
}
else
{
    app.UseCors("AllowFrontend");
    app.UseExceptionHandler("/error");
}

app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();
app.Run();
```

5. **Тестовый клиент для проверки CORS.** Для тестирования кросс-доменных запросов создан простой HTML-файл test-cors.html, который отправляет fetch-запросы к API из другого источника (http://localhost:8000).

Файл включает:

- Выбор метода запроса (GET/POST/PUT/DELETE)
- Редактируемое поле для тела запроса (JSON)
- Отображение статуса, заголовков и тела ответа
- Обработку ошибок с подсказками по CORS

## 2. Запуск приложения и тестирование

Для тестирования CORS использовался **браузерный клиент** (тестовый HTML-файл) и встроенные **DevTools** для анализа заголовков. Приложение запускалось через **Visual Studio**, тестовый файл обслуживался через локальный сервер `python -m http.server 8000` с помощью **Visual Studio Code**.

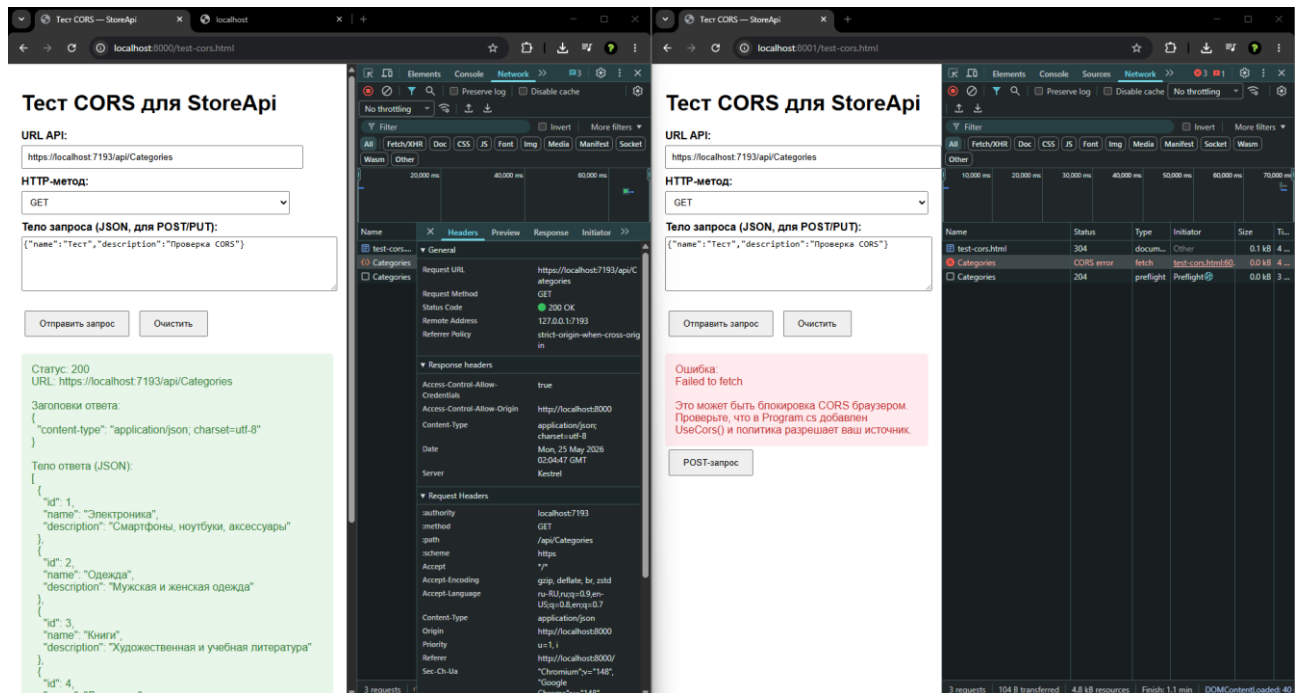
Таблица 1 Реализованные политики CORS

| Политика      | Разрешённые источники                                 | Методы                                   | Заголовки                                         |
|---------------|-------------------------------------------------------|------------------------------------------|---------------------------------------------------|
| AllowAll      | Любые (*)                                             | Любые                                    | Любые                                             |
| AllowFrontend | http://localhost:8000,<br>https://myshop-frontend.com | GET, POST,<br>PUT,<br>DELETE,<br>OPTIONS | Content-Type, Authorization, X-<br>Requested-With |

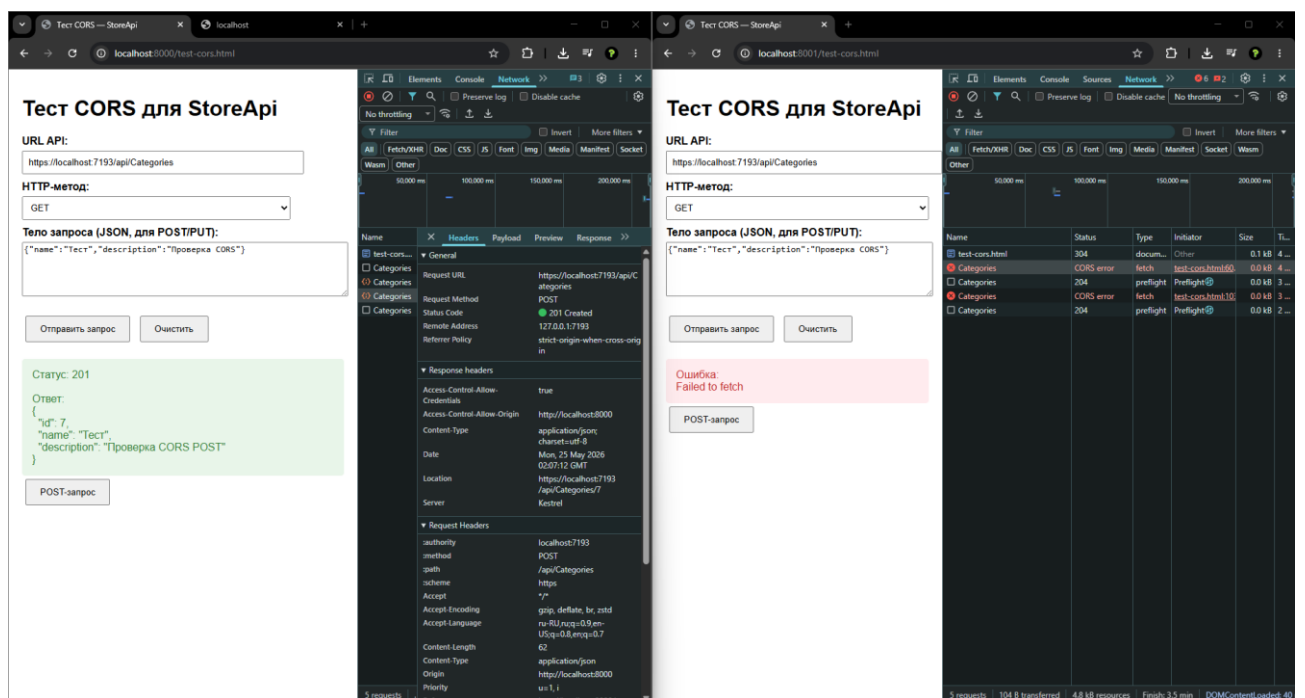
Таблица 2 Результаты тестирования запросов с политикой AllowFrontend

| № | Источник запроса      | URL API         | Метод | Результат  |
|---|-----------------------|-----------------|-------|------------|
| 1 | http://localhost:8000 | /api/Categories | GET   | Статус 200 |
| 2 | http://localhost:8000 | /api/Categories | POST  | Статус 201 |
| 3 | http://localhost:8001 | /api/Categories | GET   | CORS error |
| 4 | http://localhost:8001 | /api/Categories | POST  | CORS error |

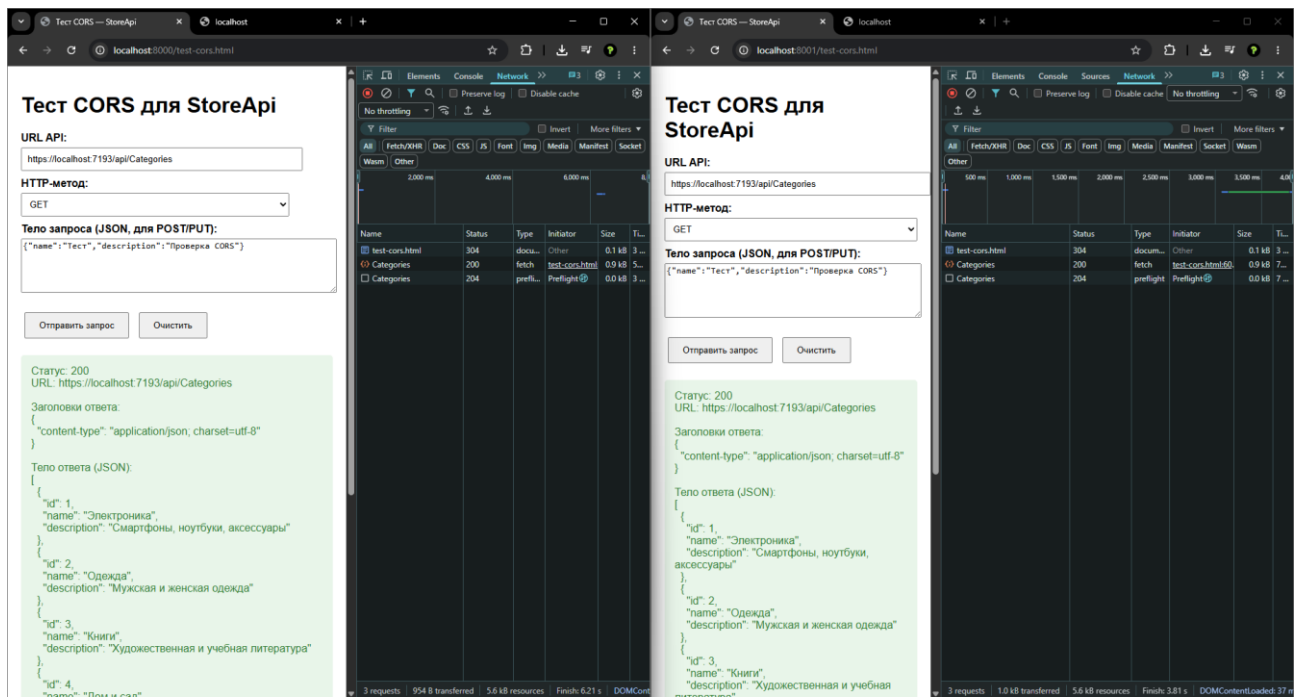
При проверке запросов с политикой AllowAll оба источника запросов не получают ошибок.



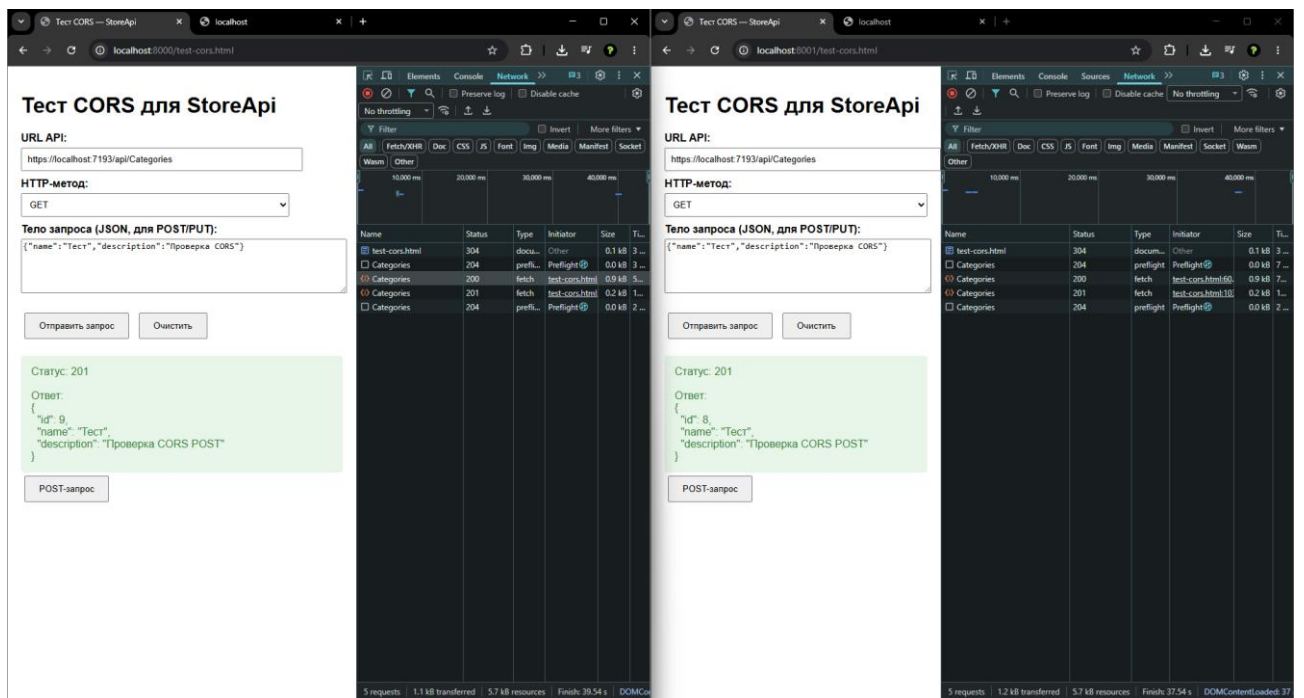
Изображение 1 GET-запрос с политикой AllowFrontend



Изображение 2 POST-запрос с политикой AllowFrontend



Изображение 3 GET-запрос с политикой AllowAll



Изображение 4 POST-запрос с политикой AllowAll

### 3. Листинг кода

#### test-cors.html (фрагмент)

```
<script>
  // Ключевая функция: отправка кросс-доменного запроса
  async function sendRequest() {
    const apiUrl = document.getElementById('apiUrl').value.trim();
    const method = document.getElementById('method').value;
    const bodyText = document.getElementById('body').value.trim();
    const resultDiv = document.getElementById('result');

    resultDiv.textContent = 'Отправка запроса...';

    try {
      // Формирование запроса к другому источнику (localhost:8000 →
      localhost:7193)
      const options = {
        method: method,
        headers: { 'Content-Type': 'application/json' } // Вызывает
      preflight OPTIONS
      };

      if (['POST', 'PUT', 'PATCH'].includes(method) && bodyText) {
        options.body = bodyText;
      }

      const response = await fetch(apiUrl, options);

      // Успешный ответ: браузер разрешает чтение благодаря заголовкам
      Access-Control-Allow-*
      let output = `Статус: ${response.status} ${response.statusText}\n`;
      output += `Заголовки
      ответа:\n${JSON.stringify(Object.fromEntries(response.headers), null, 2)}\n\n`;

      if (response.headers.get('content-type')?.includes('application/json'))
      {
        const data = await response.json();
        output += `Тело ответа:\n${JSON.stringify(data, null, 2)}`;
      } else {
        output += `Тело ответа:\n${await response.text()}`;
      }

      resultDiv.className = 'result success';
      resultDiv.textContent = output;

    } catch (error) {
      //Ошибка CORS: браузер блокирует доступ к ответу, если сервер не вернул
      разрешающие заголовки
      resultDiv.className = 'result error';
      resultDiv.textContent = `Ошибка:\n${error.message}\n\nПроверьте консоль
      браузера (F12 → Console).
      Ошибка "Access to fetch ... has been blocked by CORS policy" означает, что
      сервер не отправил
      заголовок Access-Control-Allow-Origin для данного источника.`;
      console.error('Fetch error:', error);
    }
  }
</script>
```